

Implementación en paralelo del método unilateral de Jacobi para extraer componentes principales utilizando CUDA

Proyecto semestral

Lenin Valeria Rivas

Cecilia Hernández Rivas

Facultad de Ingeniería
Universidad de Concepción

30 de abril de 2025

Tabla de contenidos

1 Introducción

2 Metodología

Tabla de contenidos

1 Introducción

2 Metodología

Contexto

- El análisis de componentes principales (PCA) es una técnica matemática utilizada para problemas de reducción de dimensionalidad, compresión de datos con pérdidas, extracción de componentes y visualización de datos.
- En esencia, es un problema de optimización, definida de dos posibles maneras, resultando en un mismo algoritmo.

Contexto

- El primero corresponde a una proyección ortogonal de los datos en un espacio de lineal de menor dimensión, en el cual la **varianza** es maximizada
- El segundo corresponde a una proyección lineal que **minimiza** el costo de proyección promedio, definido como la mean squared distance entre los datos y sus proyecciones.

Contexto

- Para aplicar el criterio de máxima varianza, se busca obtener el conjunto de vectores unitarios ordenados por la varianza proyectada sobre ello.
- Para ello, se calcula la **matriz de covarianza**, una matriz cuadrada de tamaño $n \times n$, con n el número de valores de la matriz original.

Problemas actuales

- En el marco de implementar un algoritmo de PCA computacionalmente, el cálculo de la matriz de covarianza resulta costoso en materia de memoria y tiempo de ejecución para matrices muy grandes.
- Por lo tanto, se utilizan técnicas alternativas para estimar los componentes principales de la matriz de origen.

Problemas actuales

- Entre las técnicas alternativas se encuentra la *descomposición de valores singulares* (SVD), que busca descomponer la matriz X de origen en tres submatrices, específicas:

$$X = U\Sigma V^T \quad (1)$$

- Σ corresponde a los **valores singulares** de la matriz X mientras que U y V contienen información sobre los espacios que basan a las filas y columnas de la matriz.

Problemas actuales

- El algoritmo de SVD requiere realizar un proceso de ortogonalización entre pares de columnas de manera iterativa, con un valor de tolerancia ϵ .
- Para matrices muy grandes, este proceso realizado secuencialmente puede resultar en una complejidad cercana a $\mathcal{O}(n^{1.5})$, para una matriz de tamaño $T \times D$ y $n = TD$.

Motivación

- Se propone dentro del marco del curso, analizar los efectos de implementar el algoritmo en paralelo usando CUDA, en términos de complejidad temporal.

2 Metodologia

Algoritmo de SVD

- Para una matrix $X \in \mathbb{R}^{T \times D}$, con T filas y D columnas, se busca descomponer X de la siguiente forma:

$$X = U\Sigma V^T \Leftrightarrow XV = U\Sigma \quad (2)$$

- Donde puede identificarse una nueva matrix $W = XV$. El objetivo del algoritmo es generar W y V a partir de X .
- Para ello, se pueden rotar iterativamente todos los posibles pares de columnas en X hasta que los valores no diagonales de la matrix $X^T X$ converjan a un cierto valor de tolerancia ϵ ($\mathcal{O}(D^2)$).

Algoritmo de SVD

- El proceso de ortogonalización de las columnas se realiza mediante **rotaciones de Givens**, buscando una matriz de rotación G , tal que el producto de las columnas con G sean ortogonales.

$$G = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (3)$$

- Bajo ciertas estrategias matemáticas, estos valores pueden obtenerse definiendo un parámetro especial τ , definido como:

$$\tau = \frac{x_j^T \cdot x_j - x_i^T \cdot x_j}{2(x_i^T \cdot x_j)} \quad (4)$$

- Definido por el productos punto entre las columnas a rotar (x_i, x_j) y los productos punto consigo mismas. ($\mathcal{O}(T)$)

Algoritmo de SVD

- Previo a este proceso iterativo se inicializa una matriz identidad V de tamaño $D \times D$.
- Se aplica la rotación obtenida a las columnas de X usando la matriz G , y adicionalmente se "vuelcan" acumulativamente estas rotaciones en la columnas correspondientes de V . ($\mathcal{O}(D)$).
- Una vez que se alcanza la convergencia al valor ϵ en todas las columnas, la matriz X pasa a ser W , tal que $W = XV$. En base a W pueden obtenerse tanto Σ como U .

Complejidad

- Considerando las complejidades anteriores, asumiendo un proceso secuencial con iteraciones anidadas, la complejidad de este algoritmo sería de $\mathcal{O}(n(D^2(T + D)))$, con n el número de iteraciones necesarias para alcanzar la convergencia.

Paralelización

- Dentro del algoritmo, el proceso de ortogonalización iterativo por cada par de columnas puede paralelizarse, definiendo un algoritmo de "barrido" por todos los pares de columnas y definiéndolo como un kernel para una implementación en CUDA.

Pseudocódigo

Algorithm 1 CUDA based One-Sided Jacobi implementation

```
1: Inputs:
    $X \leftarrow \text{data matrix } (T \times D);$ 
    $\epsilon \leftarrow \text{tolerance};$ 
    $\text{max\_sweeps} \leftarrow \text{maximum amount of sweeps};$ 
2: Initializations:
    $I \leftarrow \{0, \dots, D - 1\}, \text{ indexes of } X;$ 
    $V \leftarrow \text{identity matrix of size } D \times D;$ 
    $\text{any\_rots} \leftarrow \text{loop condition};$ 
3: Initialize  $X, V$  to Device;
4:  $\text{any\_rots\_host} \leftarrow \text{True};$ 
5:  $\text{sweeps} \leftarrow 0, \text{ sweep counter};$ 
6: while ( $\text{any\_rots\_host}$  and  $\text{sweeps} < \text{max\_sweeps}$ ) do
7:    $\text{any\_rots\_host} \leftarrow \text{False};$ 
8:    $\text{any\_rots\_device} \leftarrow \text{any\_rots\_host}, \text{ copy host value to device};$ 
9:   for ( $r$  from 0 to  $D - 1$ ) do
10:    Sweep  $\lll D/2$  blocks, 1 thread  $\ggg (X, V, I_D, \text{any\_rots\_device});$  execute device kernel
11:    synchronize threads;
12:    circular_shift( $I$ ), shuffle indexes using circular shift;
13:   end for
14:    $\text{any\_rots\_host} \leftarrow \text{any\_rots\_device};$ 
15:    $\text{sweeps}++;$ 
16: end while
```

Kernel propuesto

Algorithm 2 Sweep kernel

```
1: Sweep( $X, V, I_D, \text{any\_rots\_device}$ )
2: for each pair( $i, j$ ) in  $I_D$  do
3:    $a \leftarrow X[i] \cdot X[i]$ ;
4:    $b \leftarrow X[j] \cdot X[j]$ ;
5:    $c \leftarrow X[i] \cdot X[j]$ ;
6:    $\text{needs\_rot} \leftarrow (|c|/\sqrt{a \cdot b}) > \epsilon, \text{ rotation condition};$ 
7:   if  $\text{needs\_rot}$  then
8:      $\tau, t, \cos\_theta, \sin\_theta \leftarrow \text{given\_rot}(X[i], X[j]);$ 
9:      $\text{update}(X[i], X[j]);$ 
10:     $\text{update}(V[i], V[j]);$ 
11:     $\text{any\_rots\_device} = \text{any\_rots\_device} \text{ or } 1;$ 
12:   end if
13: end for
```

Algoritmo paralelo

- El algoritmo utiliza una serie de "barridos" utilizando un arreglo de índices, realizando un proceso de *circular shift*, asegurando que la repetición de pares sea mínima.
- Cada hilo realiza el proceso de rotación de manera independiente, comprobando si el par a rotar ya es ortogonal en sí mismo.
- Esta componente de paralelismo reduciría la complejidad del barrido de $\mathcal{O}(D^2)$ a $\mathcal{O}(D)$, dado que solo tienen que realizarse $D - 1$ barridos como máximo.

Bibliografía I

Implementación en paralelo del método unilateral de Jacobi para extraer componentes principales utilizando CUDA

Proyecto semestral

Lenin Valeria Rivas

Cecilia Hernández Rivas

Facultad de Ingeniería
Universidad de Concepción

30 de abril de 2025